```
"""
app.py — Spurs Event Day Parking Notifier
Run with: streamlit run app.py
"""
from datetime import datetime
import pandas as pd
import streamlit as st

import db
import scraper
from rules import notification_times,
DEFAULT_RULES

st.set_page_config(page_title="Spurs Event
Day Parking Alerts", layout="wide")
db.init_db()

CPZ_ZONES = ["NPW", "SA", "BC", "BGN",
"BGW", "SL", "7S", "Not sure / other"]

st.title("🚗 Tottenham Hotspur Stadium —
Event Day Parking Alerts")
st.caption(
    "Get an email reminder before matchday
and event-day parking restrictions "
    "start around Tottenham Hotspur
Stadium, so you can move your car and avoid
"
    "a Penalty Charge Notice."
)
```

```
tab_register, tab_upcoming, tab_admin =
st.tabs(["Register", "Upcoming events",
"Admin"])

# ----------------------------------------
---------------------------------
# Register tab
# ----------------------------------------
---------------------------------
with tab_register:
    st.subheader("Sign up for alerts")
    with st.form("register_form"):
        email = st.text_input("Email
address")
        zone = st.selectbox(
            "Your CPZ zone (check your
parking permit or a recent PCN if unsure)",
            CPZ_ZONES,
        )
        lead_time = st.selectbox(
            "Advance notice — how long
before you need to move your car?",
            [12, 24, 48, 72],
            index=1,
            format_func=lambda h: f"{h}
hours",
        )
        reminder = st.selectbox(
            "Same-day reminder — how close
to the restriction start?",
            [1, 2, 3, 4],
            index=2,
            format_func=lambda h: f"{h}
```

```
hours before",
        )
        submitted =
st.form_submit_button("Subscribe")

        if submitted:
            if not email or "@" not in
email:
                st.error("Please enter a
valid email address.")
            else:
                db.add_subscriber(email,
zone, lead_time, reminder)
                st.success(
                    f"You're subscribed!
We'll email {email} {lead_time}h before "
                    f"and again {reminder}h
before restrictions start on event days."
                )

    st.markdown("---")
    st.subheader("Unsubscribe")
    with st.form("unsub_form"):
        unsub_email = st.text_input("Email
address to remove")
        unsub_zone = st.selectbox("Zone",
CPZ_ZONES, key="unsub_zone")
        if
st.form_submit_button("Unsubscribe"):
            db.unsubscribe(unsub_email,
unsub_zone)
            st.info(f"{unsub_email} has
been unsubscribed from {unsub_zone}
alerts.")
```

```
# ----------------------------------------
---------------------------------
# Upcoming events tab (public view)
# ----------------------------------------
---------------------------------
with tab_upcoming:
    st.subheader("Upcoming stadium events")
    events = db.get_upcoming_events()

    if not events:
        st.info("No upcoming events loaded
yet. Check the Admin tab to add some.")
    else:
        rows = []
        for e in events:
            w = notification_times(
                e["event_date"],
e["kickoff_time"],
                lead_time_hours=24,
reminder_hours_before=3,
            )
            rows.append({
                "Date": e["event_date"],
                "Event": e["event_name"],
                "Type": e["event_type"],
                "Kick-off / Start":
e["kickoff_time"],
                "Move car by (est.)":
w["move_by"].strftime("%a %d %b, %H:%M"),
                "Restrictions clear
(est.)":
w["restriction_end"].strftime("%H:%M"),
            })
```

```
        st.dataframe(pd.DataFrame(rows),
use_container_width=True, hide_index=True)
        st.caption(
            "Estimated times use
conservative defaults (restrictions from "
            f"
{DEFAULT_RULES['closure_starts_hours_before
']}h before kick-off to "
            f"
{DEFAULT_RULES['restriction_ends_hours_afte
r']}h after). Haringey "
            "Council confirms exact bay
suspension times ~7 days before each event
— "
            "always check officially nearer
the date."
        )

# ----------------------------------------
---------------------------------
# Admin tab
# ----------------------------------------
---------------------------------
with tab_admin:
    st.subheader("Pull candidate events
from tottenhamhotspurstadium.com")
    st.caption(
        "Scrapes the official events pages
for concerts/NFL/other stadium events "
        "(not football fixtures — those
need a separate source, see prompt.md). "
        "Dates are usually available but
start TIMES are not, so nothing here "
        "reaches subscribers until you
```

```
review and confirm it below."
    )
    if st.button("Run scraper now"):
        with st.spinner("Fetching
tottenhamhotspurstadium.com..."):
            try:
                new_count =
scraper.scrape_events()
                st.success(f"Found
{new_count} new candidate event(s).")
            except Exception as e:
                st.error(f"Scraper failed:
{e}")

    pending =
db.get_pending_scraped_events()
    if pending:
        st.write(f"**{len(pending)}
candidate(s) awaiting review:**")
        for p in pending:
            with
st.form(f"review_{p['id']}"):
                c1, c2, c3, c4 =
st.columns([2, 3, 2, 2])
                c1.write(p["event_date"])
                c2.write(p["event_name"])
                kickoff =
c3.time_input("Start time",
key=f"kt_{p['id']}")
                etype = c4.selectbox(
                    "Type", ["concert",
"nfl", "boxing", "other"],
key=f"et_{p['id']}"
                )
```

```
                bc1, bc2 = st.columns(2)
                approve =
bc1.form_submit_button("Approve & publish")
                reject =
bc2.form_submit_button("Reject")

                if approve:

db.approve_scraped_event(p["id"],
kickoff.strftime("%H:%M"), etype)
                    st.success(f"Published:
{p['event_name']} on {p['event_date']}")
                    st.rerun()
                if reject:

db.reject_scraped_event(p["id"])
                    st.info(f"Rejected:
{p['event_name']}")
                    st.rerun()
    else:
        st.caption("No pending candidates.
Run the scraper above to check for new
events.")

    st.markdown("---")
    st.subheader("Add an event manually")
    st.caption(
        "Use this for football fixtures
(not covered by the scraper) or to add "
        "anything the scraper missed."
    )
    with st.form("add_event_form"):
        c1, c2 = st.columns(2)
        event_date = c1.date_input("Event
```

```
date")
        kickoff = c2.time_input("Kick-off /
start time")
        event_name = st.text_input("Event
name", placeholder="e.g. Spurs vs Arsenal")
        event_type = st.selectbox("Event
type", ["football", "concert", "nfl",
"boxing", "other"])
        notes = st.text_area("Notes
(optional)")

        if st.form_submit_button("Add
event"):
            db.add_event(
                event_date.strftime("%Y-%m-
%d"),
                event_name,
                event_type,
                kickoff.strftime("%H:%M"),
                notes,
            )
            st.success(f"Added:
{event_name} on {event_date}")
            st.rerun()

    st.markdown("---")
    st.subheader("Manage events")
    events = db.get_upcoming_events()
    if events:
        for e in events:
            c1, c2, c3, c4, c5 =
st.columns([2, 3, 2, 2, 1])
            c1.write(e["event_date"])
            c2.write(e["event_name"])
```

```
            c3.write(e["event_type"])
            c4.write(e["kickoff_time"])
            if c5.button("Delete",
key=f"del_{e['id']}"):
                db.delete_event(e["id"])
                st.rerun()
    else:
        st.info("No events yet.")

    st.markdown("---")
    st.subheader("Subscribers")
    subs = db.get_active_subscribers()
    st.write(f"**{len(subs)}** active
subscriber(s)")
    if subs:
        st.dataframe(pd.DataFrame(subs)
[["email", "zone", "lead_time_hours",
"reminder_hours_before"]],

use_container_width=True, hide_index=True)
```